

Name: Prajwal Tanjal Class: _____ Batch: A
Subject: PL Roll No.: 21 Exp. No.: 2
Name of the Experiment: _____
Performed on: _____
Submitted on: _____

Marks	Teacher's Signature with date
<u>10</u>	<u>[Signature]</u> <u>12/12/21</u>

P.L Assignment 2

1. List of Python sequence data type and state one key characteristic at each with respect to mutability or ordering.

→ data type are: String (str): Immutable and ordered collection of characters

- o list: Mutable & ordered collection of elements
- o Tuple: Immutable and ordered collection of elements
- o Range: Immutable and ordered sequence of numbers.

2. Explain how indexing and slicing work in Python sequences, illustrate how they differ when applied to strings & lists.

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐
Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐
Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

[Signature]
Teacher's Signature

→ Indexing is used to access a single element of a sequence using its position. Positive indexing starts from 1

Slicing extracts a part of sequence using the syntax.

Difference in strings & lists:

o In strings, indexing / slicing returns characters or substrings

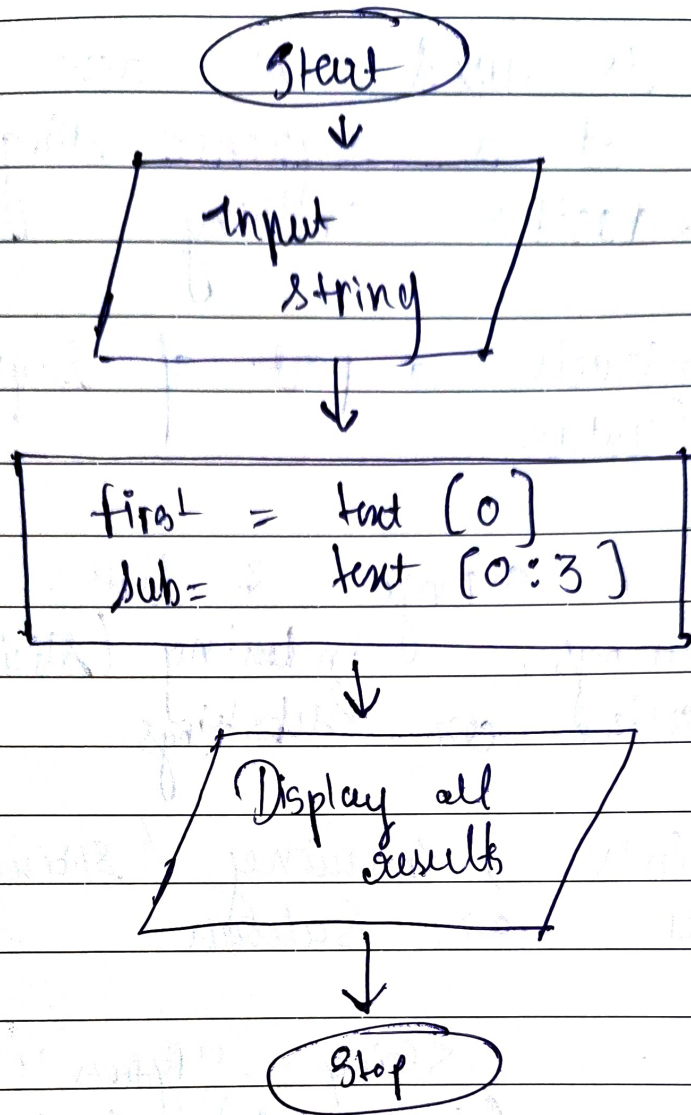
o In lists, indexing / slicing returns elements or sublists.

example : String : - "Python" [1:4] → "yth"
list : (1, 2, 3, 4) [1:3] → [2, 3]

3 Python program using indexing, slicing & built in functions.

→ Code : text = "engineering"
first = char = text[0]
sub-text = text[0:5]
length = len(text)
upper-text = text.upper()

Print ("first character:", first - char)
Print ("Sliced text:", sub-text)
Print ("length:", length)
Print ("upper case:", upper-text)



4] Difference between equality (==) & Identity (is)

→ == (Equality operator) checks whether two sequences refer to the same object in memory.

eg. $a = [1, 2]$
 $b = [1, 2] \rightarrow a == b$ is True,
but a is false.

5

Evaluation of lists, sets & dictionaries for student records.

-
- lists allow duplicates and are ordered, but searching is slower.
 - sets store only unique values & are fast for membership testing.

Most efficient :- Dictionaries are best for student records because each student can be stored using unique ID as a key, enabling fast look up, update & retrieval.

Program to Manage student data.

→

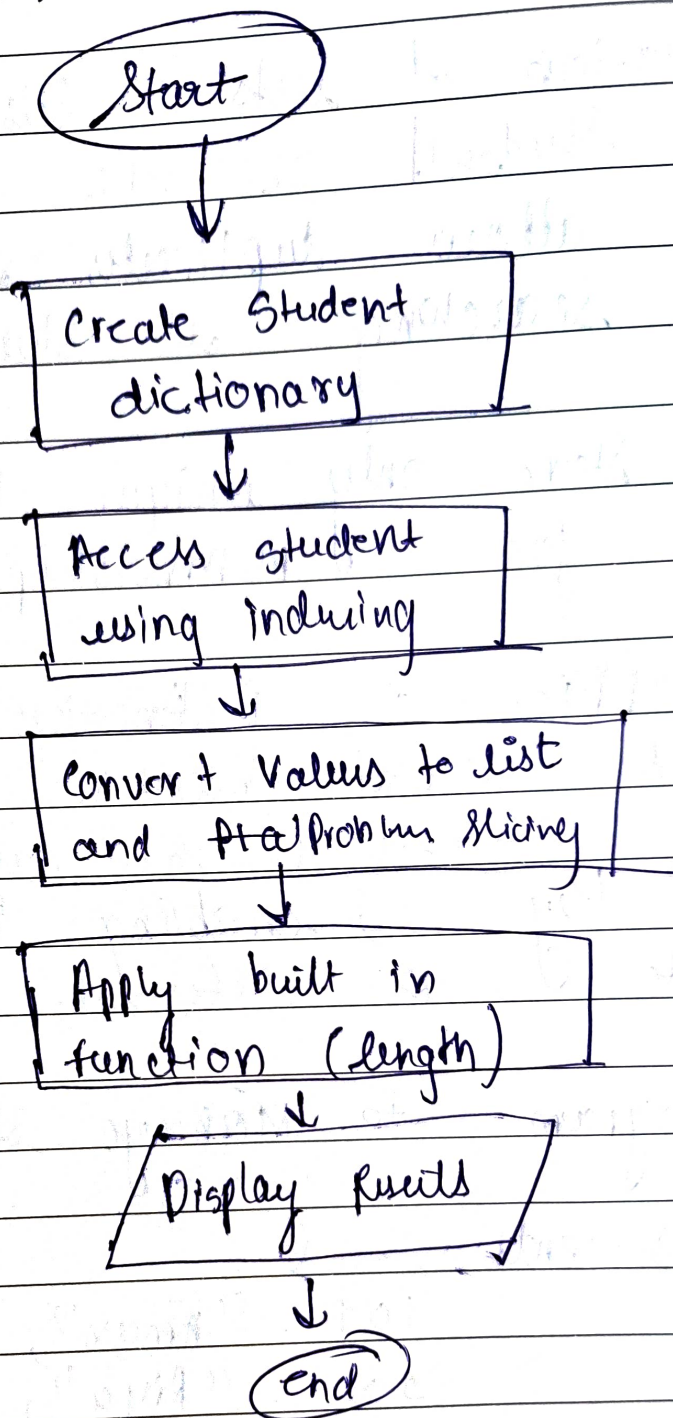
```
students = {
    101 : "Aman",
    102 : "Riya",
    103 : "Karen",
}
```

```
print ("Student with ID 102 : " + students[102])
```

```
names = list(students.values())
```

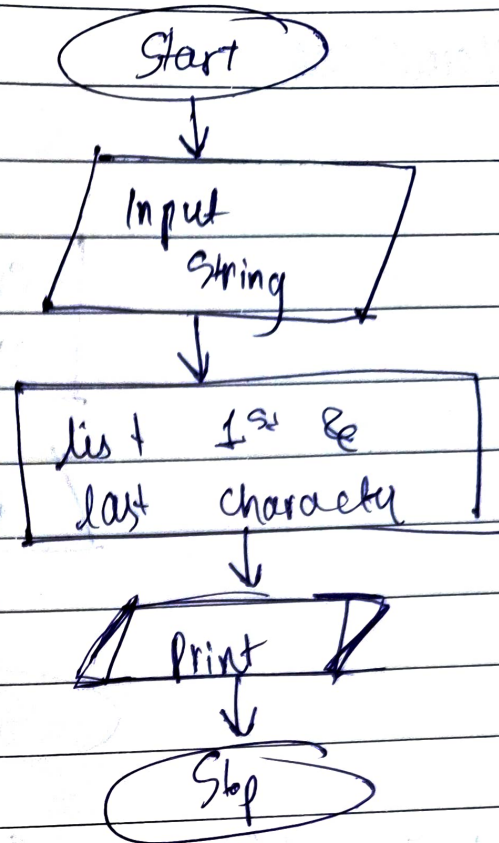
```
print ("First two students : ", names[0:2])
```

```
print ("Total students : ", len(students))
```

- 7 Print 1st & last Character of a string.
- Algorithm :
1. Start
 2. Input a string
 3. Print 1st character using index
 4. Print last character using index - 1
 5. Stop.

flowchart :-

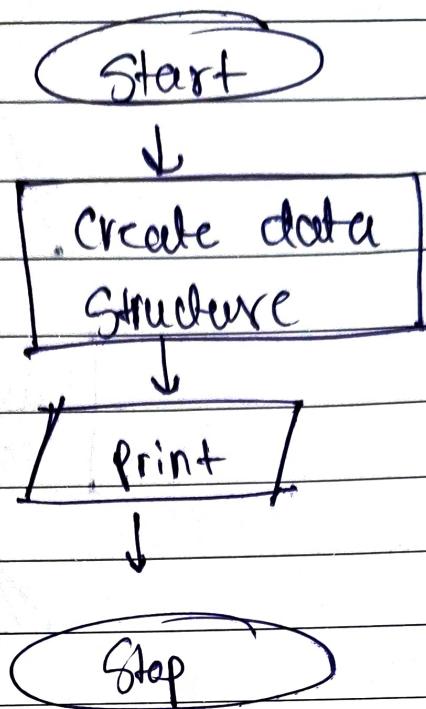


output :- enter a string : Python
first character : P
last character : n.

8 Create Create list, tuple, set & dictionary.

→ Algorithm : 1. Start
2. Create list, tuple, set dictionary
3. Display them
4. stop.

flowchart



Code: $lst = [1, 2, 3]$
 $tpl = (4, 5, 6)$
 $st = \{7, 8, 9\}$
 $dict = \{ 'a': 1, 'b': 2 \}$

Output = $list :- [1, 2, 3]$
 $tuple :- (4, 5, 6)$
 $set :- \{7, 8, 9\}$
 $Dictionary :- \{ 'a': 1, 'b': 2 \}$

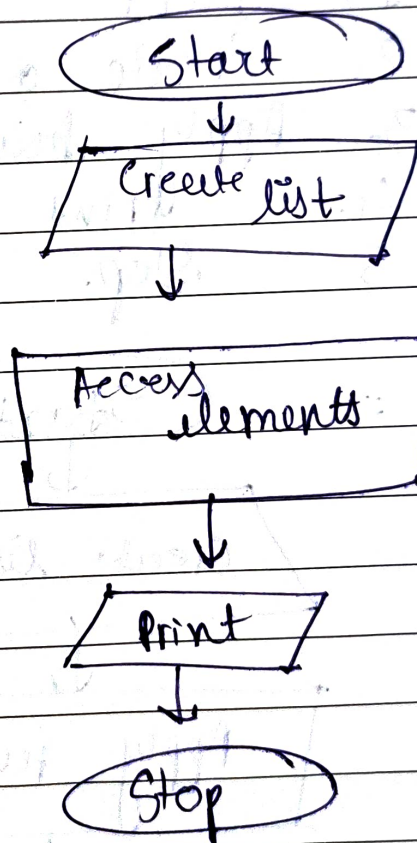
9

Program to understand indexing

Algorithm:

1. Start
2. Create a list
3. Access elements using indexing
4. Print elements
5. Stop.

Flowchart :



Code: `numbers = [10, 20, 30, 40]`
`Print ("first element:", numbers[0])`
`Print ("second element:", numbers[1])`
`Print ("last element:", numbers[-1])`

output

first element : 10

Second element : 20

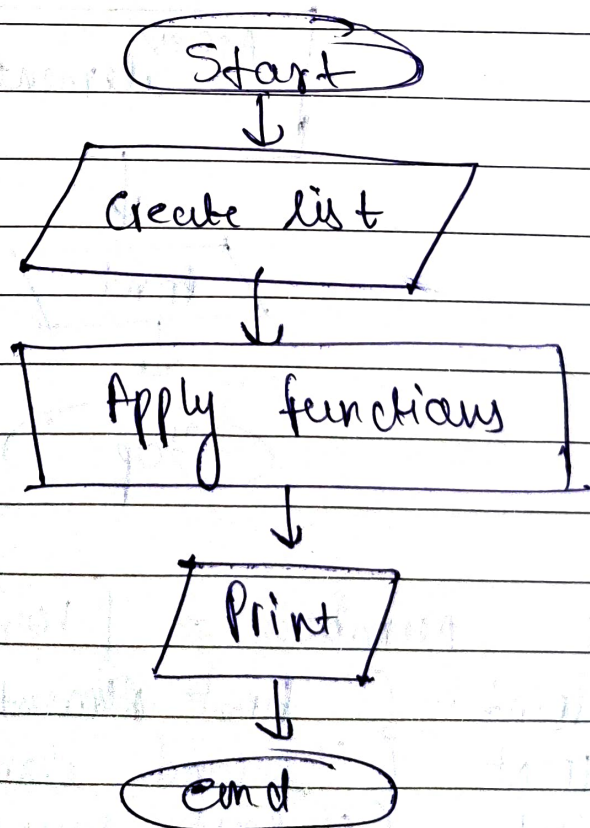
Third element : 30

10] Program to understand built in function.

→ Algorithm.

1. Start
2. Create a list
3. Apply built in functions
4. Print results
5. Stop.

flowchart :-



code :- numbers = [5, 10, 15, 20]

```
print ("length", len (numbers))  
print ("Maximum : ", Max (numbers))  
print ("Minimum : ", min (numbers))  
print ("Sum : " sum, (numbers))
```

output :- length = 4
Max = 20
Min = 5
sum = 50.